

规划模型建模规范

线性规划、整数规划、0-1 规划、蒙特卡洛、非线性规划、目标规划与现代优化方法

适用于数学建模论文与代码实现

2026 年 7 月 8 日

摘要

规划模型的核心不是“评价谁更好”，而是在一组约束条件下选择决策变量，使目标函数达到最优或尽量满足多个目标。本文把常见规划模型整理为统一套路：先把实际问题转化为决策变量、目标函数和约束条件，再根据目标和约束的形式选择线性规划、整数规划、非线性规划、目标规划或随机模拟方法。文档包含基本概念、适用条件、数学原理、建模步骤、论文结论评价模板、扩展方法和 Python 代码样式。

目录

1 规划模型的总主旨	3
1.1 论文中的总建模步骤	3
2 规划模型分类总表	3
3 线性规划	4
3.1 概念与标准形式	4
3.2 使用条件	4
3.3 建模模板	5
3.4 Python 代码样式：SciPy linprog	5
4 整数规划与 0-1 规划	5
4.1 整数规划	5
4.2 0-1 规划	5
4.3 常用建模技巧	6
4.4 Python 代码样式：OR-Tools CP-SAT 0-1 背包	6
5 蒙特卡洛法	7
5.1 它是不是规划模型？	7
5.2 建模步骤	7
5.3 Python 代码样式：库存订货的模拟优化	7
6 非线性规划	8
6.1 无约束极值	8
6.2 等式约束：拉格朗日乘子法	8
6.3 不等式约束：KKT 条件	8

目录	2
6.4 Python 代码样式: SLSQP 求有约束非线性规划	8
7 目标规划	9
7.1 为什么需要目标规划	9
7.2 加权系数法	9
7.3 优先等级法	9
7.4 Chebyshev 目标规划	9
7.5 Python 代码样式: 加权目标规划	10
8 现代常用方法与补充	10
8.1 凸优化	10
8.2 二次规划 QP	10
8.3 动态规划	10
8.4 网络流与路径优化	10
8.5 随机规划与鲁棒优化	11
8.6 启发式与元启发式	11
8.7 机器学习与优化结合	11
9 如何根据题目选择模型	11
10 论文写作模板	12
10.1 模型建立段落	12
10.2 结果分析段落	12
10.3 局限性段落	12
11 完整代码模板	12
12 OR-Tools 0-1 规划代码	13
13 CVXPY 凸优化代码	14
14 参考资料与网页实例	14

1 规划模型的总主旨

一句话概括

规划问题的主旨是：在资源、规则、容量、时间、逻辑、风险等限制条件下，寻找一组可行决策变量，使某个目标最大、最小，或使多个目标尽可能达到要求。

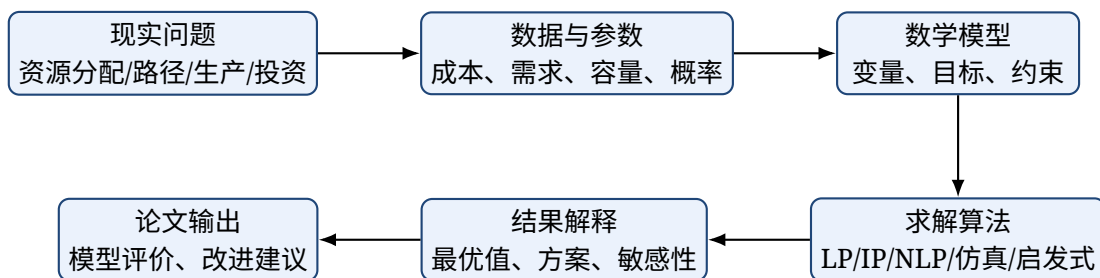
和 TOPSIS、FCE、熵权法、灰色关联等综合评价模型相比，规划模型更强调“决策”。评价模型通常回答“哪个对象更好”；规划模型通常回答“应该怎么分配、怎么安排、怎么选择”。因此它的论文写法要围绕以下三件事展开：

决策变量 目标函数 约束条件

一般规划模型可写为

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p, \\ & x \in X. \end{aligned} \quad (1)$$

其中 x 是决策变量， $f(x)$ 是目标函数， g_i, h_j 是约束， X 表示变量类型限制，例如连续变量、整数变量、0-1 变量等。



1.1 论文中的总建模步骤

1. **明确决策目标。**是最小成本、最大收益、最短时间、最低风险，还是多个目标共同满足？
2. **定义决策变量。**变量是连续数量、整数数量、是否选择、路径顺序，还是随机样本？
3. **建立目标函数。**把“最好”写成数学表达式，例如 $\min c^T x$ 或 $\max p^T x$ 。
4. **写出约束条件。**资源、容量、时间、逻辑、供需平衡、预算、政策要求都要转成等式或不等式。
5. **判断模型类别。**线性、整数、0-1、非线性、随机、目标规划、多目标规划等。
6. **选择求解方法。**线性规划可用单纯形法或内点法，整数规划可用分支定界/割平面/CP-SAT，非线性规划可用梯度、牛顿、SLSQP、trust-constr 或全局优化方法。
7. **解释结果。**包括最优方案、最优值、约束是否紧、敏感性分析、模型局限和改进建议。

2 规划模型分类总表

模型类别	标准形式	适合问题	常见方法
线性规划 LP	$\min c^T x, Ax \leq b$	资源分配、生产计划、运输、配料、投资组合线性近似	单纯形法、内点法、SciPy linprog、OR-Tools、Pyomo
整数规划 IP	LP 加 $x_j \in \mathbb{Z}$	人数、台数、车辆数、批次等必须为整数	分支定界、割平面、分支割、CP-SAT
0-1 规划 BIP	$x_j \in \{0, 1\}$	选或不选、开或不开、是否分配、路径边是否采用	大 M 法、逻辑约束、背包、指派、选址、CP-SAT
蒙特卡洛法	随机抽样估计 $E[g(X)]$	不确定性模拟、风险分析、随机参数、难以解析的问题	随机采样、情景模拟、置信区间、随机搜索
非线性规划 NLP	$\min f(x), f, g, h$ 可非线性	成本曲线、收益递减、物理模型、机器学习参数	梯度法、牛顿法、拉格朗日、KKT、SLSQP、trust-constr
目标规划 GP	最小化目标偏差变量	多目标同时满足、目标有期望值、指标有优先级	加权目标规划、优先等级法、Chebyshev 目标规划
凸优化	f, g_i 凸, h_j 仿射	大规模可靠求解、稀疏回归、投资组合、控制	CVXPY、内点法、ADMM、近端梯度
随机/鲁棒优化	参数不确定	需求、价格、天气、风险不确定	情景规划、机会约束、鲁棒集、两阶段规划
启发式/元启发式	不要求解析可导	旅行商、排班、路径、复杂组合优化	遗传算法、模拟退火、粒子群、差分进化、贝叶斯优化

3 线性规划

3.1 概念与标准形式

线性规划是最基础的规划模型。若目标函数和约束条件都是决策变量的一次函数，则可写成：

$$\begin{aligned}
 \min \quad & c^T x \\
 \text{s.t.} \quad & Ax \leq b, \\
 & A_{eq}x = b_{eq}, \\
 & l \leq x \leq u.
 \end{aligned} \tag{2}$$

若题目是最大化，可以通过 $\max z = c^T x$ 转化为 $\min -c^T x$ 。

3.2 使用条件

- **线性比例性：**每增加一单位变量，对目标和约束的影响是固定的。
- **可加性：**总成本、总收益、总资源消耗可以分项相加。
- **确定性：**参数 c, A, b 在模型期内视为已知。
- **连续性：**变量可取连续值。若必须为整数，就转为整数规划。

3.3 建模模板

1. 设变量 x_j 表示第 j 类活动的数量。
2. 写目标：最小化总成本 $\min \sum_j c_j x_j$ 或最大化总收益 $\max \sum_j p_j x_j$ 。
3. 写约束：资源限制 $\sum_j a_{ij} x_j \leq b_i$ ，需求约束 $\sum_j d_{ij} x_j \geq q_i$ 。
4. 加非负约束 $x_j \geq 0$ 。
5. 求解后说明哪些约束是紧约束，哪些资源有剩余。

3.4 Python 代码样式：SciPy linprog

```

1 import numpy as np
2 from scipy.optimize import linprog
3
4 # min 3x1 + 5x2
5 # s.t. 2x1 + x2 <= 100
6 #      x1 + 2x2 <= 80
7 #      x1, x2 >= 0
8 c = np.array([3, 5])
9 A_ub = np.array([[2, 1], [1, 2]])
10 b_ub = np.array([100, 80])
11 bounds = [(0, None), (0, None)]
12
13 res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")
14 print(res.success, res.fun, res.x)

```

4 整数规划与 0-1 规划

4.1 整数规划

整数规划在线性规划基础上增加整数约束：

$$\begin{aligned}
 \min \quad & c^T x \\
 \text{s.t.} \quad & Ax \leq b, \\
 & x_j \in \mathbb{Z}, \quad j \in I.
 \end{aligned} \tag{3}$$

如果只有部分变量要求整数，称为混合整数规划（MIP/MILP）。

整数规划适合变量本身不能拆分的情形，例如机器台数、员工人数、车辆数、课程安排、货物批次数。它通常比线性规划难得多，因为可行域不再是连续多面体，而是许多离散点。

4.2 0-1 规划

0-1 规划是整数规划的特殊形式：

$$x_j \in \{0, 1\}.$$

它最适合表达“是否”类决策：是否选择项目、是否建设仓库、是否把任务分配给某人、路径中是否经过某条边。

4.3 常用建模技巧

0-1 变量的常见含义

- $x_j = 1$ 表示选择第 j 个项目, $x_j = 0$ 表示不选择。
- $y_{ij} = 1$ 表示任务 i 分配给人员 j , 否则为 0。
- $z_k = 1$ 表示第 k 个仓库开启, 否则关闭。

1. 背包模型:

$$\max \sum_j v_j x_j, \quad \sum_j w_j x_j \leq W, \quad x_j \in \{0, 1\}.$$

2. 指派模型:

$$\min \sum_i \sum_j c_{ij} x_{ij}, \quad \sum_j x_{ij} = 1, \quad \sum_i x_{ij} = 1.$$

3. 大 M 法: 若 $y = 0$ 时要求 $x = 0$, $y = 1$ 时允许 x 取正, 可写

$$0 \leq x \leq My, \quad y \in \{0, 1\}.$$

4. 逻辑约束: 若“选择 A 则必须选择 B”, 可写 $x_A \leq x_B$ 。

4.4 Python 代码样式: OR-Tools CP-SAT 0-1 背包

```

1 from ortools.sat.python import cp_model
2
3 values = [10, 8, 15, 4]
4 weights = [5, 4, 8, 3]
5 capacity = 12
6 n = len(values)
7
8 model = cp_model.CpModel()
9 x = [model.NewBoolVar(f"x{j}") for j in range(n)]
10
11 model.Add(sum(weights[j] * x[j] for j in range(n)) <= capacity)
12 model.Maximize(sum(values[j] * x[j] for j in range(n)))
13
14 solver = cp_model.CpSolver()
15 status = solver.Solve(model)
16
17 if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
18     print("objective =", solver.ObjectiveValue())
19     print([solver.Value(x[j]) for j in range(n)])

```

5 蒙特卡洛法

5.1 它是不是规划模型？

蒙特卡洛法本身不是传统意义上的“规划模型”，而是一种随机模拟方法。它经常与规划模型结合，用来处理参数不确定、风险评估或复杂模型无法解析求解的情况。其主旨是：用大量随机样本近似期望、概率、风险或目标函数分布。

例如需求 D 、价格 P 、成本 C 不确定时，可以设定它们的概率分布，重复抽样并计算利润：

$$\Pi = P \cdot \min(Q, D) - CQ.$$

通过模拟得到 $E(\Pi)$ 、亏损概率、最坏 5% 情况等。

5.2 建模步骤

1. 确定随机变量，例如需求、价格、故障率、服务时间。
2. 选择概率分布，例如正态、三角、均匀、泊松、经验分布。
3. 生成大量随机样本。
4. 对每个样本计算目标值或约束是否满足。
5. 得到均值、方差、置信区间、风险概率和敏感性排序。
6. 如果需要优化，则在不同决策方案下重复模拟，选择期望收益高或风险小的方案。

5.3 Python 代码样式：库存订货的模拟优化

```
1 import numpy as np
2
3 rng = np.random.default_rng(0)
4 N = 20000
5 price = 12
6 cost = 7
7 salvage = 2
8
9 # demand follows a truncated normal-like sample
10 demand = np.maximum(0, rng.normal(loc=100, scale=20, size=N))
11
12 def simulate_profit(Q):
13     sales = np.minimum(Q, demand)
14     leftover = np.maximum(Q - demand, 0)
15     profit = price * sales + salvage * leftover - cost * Q
16     return profit.mean(), np.percentile(profit, 5)
17
18 candidates = np.arange(50, 151)
19 records = [(Q, *simulate_profit(Q)) for Q in candidates]
20 best = max(records, key=lambda t: t[1])
21 print("best by expected profit:", best)
```

6 非线性规划

6.1 无约束极值

无约束非线性规划写为

$$\min_{x \in \mathbb{R}^n} f(x).$$

若 f 可导，则一阶必要条件为

$$\nabla f(x^*) = 0.$$

若 Hessian 矩阵 $\nabla^2 f(x^*)$ 正定，则 x^* 是严格局部极小点；若负定，则是局部极大点；若不定，则可能是鞍点。

6.2 等式约束：拉格朗日乘子法

若问题为

$$\min f(x), \quad h_i(x) = 0,$$

构造拉格朗日函数

$$L(x, \lambda) = f(x) + \sum_i \lambda_i h_i(x).$$

候选极值点满足

$$\nabla_x L(x, \lambda) = 0, \quad h_i(x) = 0.$$

拉格朗日乘子 λ_i 可以理解为约束右端项变化时目标最优值的边际变化。

6.3 不等式约束：KKT 条件

若问题为

$$\min f(x), \quad g_i(x) \leq 0, \quad h_j(x) = 0,$$

KKT 条件为

$$\nabla f(x^*) + \sum_i \mu_i \nabla g_i(x^*) + \sum_j \lambda_j \nabla h_j(x^*) = 0, \quad (4)$$

$$g_i(x^*) \leq 0, \quad h_j(x^*) = 0, \quad (5)$$

$$\mu_i \geq 0, \quad (6)$$

$$\mu_i g_i(x^*) = 0. \quad (7)$$

其中 $\mu_i g_i(x^*) = 0$ 叫互补松弛：一个不等式约束要么紧，要么对应乘子为 0。

6.4 Python 代码样式：SLSQP 求有约束非线性规划

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 # min (x-1)^2 + (y-2)^2
5 # s.t. x + y >= 2, x^2 + y^2 <= 10
6
```

```

7 def obj(v):
8     x, y = v
9     return (x - 1)**2 + (y - 2)**2
10
11 constraints = [
12     {"type": "ineq", "fun": lambda v: v[0] + v[1] - 2},
13     {"type": "ineq", "fun": lambda v: 10 - v[0]**2 - v[1]**2},
14 ]
15 res = minimize(obj, x0=np.array([0.0, 0.0]), constraints=constraints,
16               method="SLSQP")
17 print(res.success, res.fun, res.x)

```

7 目标规划

7.1 为什么需要目标规划

实际问题常常不只有一个目标。例如企业希望成本低、产量高、污染少、员工加班少。若强行把所有目标合成一个目标，权重很难解释；若逐个优化，目标之间又可能冲突。目标规划的思想是：给每个目标设置期望值，再最小化对目标的偏离程度。

设第 k 个目标的期望值为 G_k ，目标表达式为 $f_k(x)$ ，引入正偏差 d_k^+ 和负偏差 d_k^- ：

$$f_k(x) + d_k^- - d_k^+ = G_k, \quad d_k^+, d_k^- \geq 0. \quad (8)$$

其中 d_k^- 表示未达到目标的不足量， d_k^+ 表示超过目标的超出量。根据实际含义，只惩罚不希望出现的偏差。

7.2 加权系数法

把不同目标偏差加权求和：

$$\min Z = \sum_k \alpha_k d_k^- + \beta_k d_k^+. \quad (9)$$

适合目标优先级差别不绝对、可以进行权衡的情况。

7.3 优先等级法

若目标有绝对优先级，例如安全目标高于成本目标，高优先级目标的任何改善都比低优先级目标重要，则使用优先等级法。可按优先级分层：

$$P_1 \gg P_2 \gg P_3.$$

求解时通常先最小化第一级偏差，固定第一级最优值后，再优化第二级偏差，以此类推。这也叫词典序或预 emptive 目标规划。

7.4 Chebyshev 目标规划

若希望避免某个目标偏差特别大，可最小化最大归一化偏差：

$$\min t, \quad \frac{d_k}{s_k} \leq t, \quad k = 1, \dots, K. \quad (10)$$

它强调目标之间的均衡。

7.5 Python 代码样式：加权目标规划

```

1 import numpy as np
2 from scipy.optimize import linprog
3
4 # Example: production x1, x2.
5 # Goals: profit >= 100, labor <= 40.
6 # profit = 6x1 + 8x2, labor = 2x1 + 3x2.
7 # Variables: x1,x2,d1_minus,d1_plus,d2_minus,d2_plus
8
9 c = np.array([0, 0, 5, 0, 0, 2]) # penalize profit shortfall and labor excess
10
11 A_eq = np.array([
12     [6, 8, 1, -1, 0, 0], # profit + d- - d+ = 100
13     [2, 3, 0, 0, 1, -1], # labor + d- - d+ = 40
14 ])
15 b_eq = np.array([100, 40])
16
17 bounds = [(0, None)] * 6
18 res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
19 print(res.success, res.fun, res.x)

```

8 现代常用方法与补充

8.1 凸优化

如果目标函数是凸函数，约束集合是凸集，则局部最优就是全局最优。许多看似非线性的模型其实属于凸优化，例如最小二乘、岭回归、Lasso、二次规划、二阶锥规划、部分投资组合模型。凸优化的优点是理论和算法都比较成熟，求解可靠性强。

8.2 二次规划 QP

二次规划常见形式为

$$\min \frac{1}{2}x^T Qx + c^T x, \quad Ax \leq b.$$

若 $Q \succeq 0$ ，则为凸二次规划。它常用于投资组合、最小方差、轨迹平滑、支持向量机等。

8.3 动态规划

动态规划适合多阶段决策问题。其核心是状态转移和 Bellman 最优性原理：

$$V_t(s) = \min_a \{c_t(s, a) + V_{t+1}(s')\}.$$

适用于库存控制、最短路径、设备更新、背包、资源分配等问题。

8.4 网络流与路径优化

最短路、最大流、最小费用流、运输问题、车辆路径问题都是规划模型的重要分支。若题目中出现“节点、边、路径、流量、容量”，应优先考虑网络优化模型。

8.5 随机规划与鲁棒优化

随机规划把不确定参数表示为随机变量或情景集合，通常优化期望成本或风险指标。鲁棒优化不假设准确概率分布，而是假设参数落在不确定集合中，并要求方案在最坏情况下仍可行或表现较好。

8.6 启发式与元启发式

当模型过于复杂、变量很多、非凸、离散、难以获得精确最优解时，可以使用启发式方法：

- 遗传算法 GA：适合编码清楚、可行解容易生成的问题。
- 模拟退火 SA：适合跳出局部最优。
- 粒子群 PSO：适合连续变量黑箱优化。
- 差分进化 DE：适合边界已知的全局优化。
- 贝叶斯优化：适合每次评估代价高的黑箱函数，例如机器学习超参数。

8.7 机器学习与优化结合

现代应用中经常出现“预测 + 优化”。先用机器学习预测需求、价格、风险、时间，再把预测值放入规划模型。更进一步的 end-to-end predict-then-optimize 会直接以最终决策质量训练预测模型。建模论文中若时间有限，可写成扩展思路，不必全部实现。

9 如何根据题目选择模型

题目关键词	优先模型	判断依据
成本最低、收益最大、资源限制、变量连续	线性规划	目标和约束都是一次式
人数、台数、批次、班次必须整数	整数规划	变量不可拆分
选或不选、开或不开、是否分配	0-1 规划	逻辑选择和组合决策
路径、车辆、配送、旅行商	网络优化/车辆路径/整数规划	节点和边结构明显
风险、不确定、随机需求	蒙特卡洛/随机规划/鲁棒优化	参数不是固定数
收益递减、成本曲线、非线性物理关系	非线性规划	目标或约束含乘积、幂、指数、对数等
多个目标有期望值	目标规划	目标不一定最优，而是尽量接近目标值
目标之间有绝对优先级	优先等级目标规划	高优先级目标不可牺牲
函数黑箱、不可导、局部最优多	元启发式/全局优化	解析条件难写，数值评估可行

10 论文写作模板

10.1 模型建立段落

可直接改写

为解决该资源配置问题，本文建立规划模型。设 x_j 表示第 j 类决策活动的实施水平，根据问题目标构造目标函数 $f(x)$ ，并将预算、容量、需求、逻辑关系和变量取值范围转化为约束条件。若目标函数和约束均为线性形式，则采用线性规划；若部分变量具有整数或是否选择含义，则进一步建立整数规划或 0-1 规划。模型求解后，根据最优解、目标函数值、约束紧性和敏感性分析评价方案的可行性与稳定性。

10.2 结果分析段落

结论评价模板

由求解结果可知，模型在满足全部约束条件的前提下取得最优目标值 Z^* ，对应决策方案为 x^* 。从变量取值看，资源主要分配给了……，说明这些活动在目标函数中具有较高收益或较低成本。从约束松弛量看，……约束达到边界，属于限制系统性能的关键约束；……资源仍有剩余，对最优值影响较小。若对关键参数进行扰动，最优方案变化不大，说明模型结论具有一定稳定性。

10.3 局限性段落

常见局限写法

本文模型将部分参数视为确定值，未完全考虑需求波动、价格变化和执行过程中的随机扰动；同时，线性化处理可能简化了真实的非线性关系。后续可引入随机规划、鲁棒优化或蒙特卡洛模拟，对不确定性下的方案表现进行进一步检验。

11 完整代码模板

以下代码把常用规划模型的写法放在一个文件中。实际使用时不需要全用，只选与题目匹配的部分。

```
1 import numpy as np
2 from scipy.optimize import linprog, minimize, differential_evolution
3
4 # ----- Linear Programming -----
5 def solve_lp():
6     c = np.array([3, 5])
7     A_ub = np.array([[2, 1], [1, 2]])
8     b_ub = np.array([100, 80])
9     res = linprog(c, A_ub=A_ub, b_ub=b_ub,
```

```

10         bounds=[(0, None), (0, None)], method="highs")
11     return res
12
13 # ----- Nonlinear Programming -----
14 def solve_nlp():
15     def f(v):
16         x, y = v
17         return (x - 1)**2 + (y - 2)**2
18     cons = [
19         {"type": "ineq", "fun": lambda v: v[0] + v[1] - 2},
20         {"type": "ineq", "fun": lambda v: 10 - v[0]**2 - v[1]**2},
21     ]
22     return minimize(f, x0=np.array([0.0, 0.0]), constraints=cons, method="
SLSQP")
23
24 # ----- Monte Carlo Simulation -----
25 def monte_carlo_inventory(seed=0):
26     rng = np.random.default_rng(seed)
27     demand = np.maximum(0, rng.normal(100, 20, size=20000))
28     price, cost, salvage = 12, 7, 2
29     def profit(Q):
30         sales = np.minimum(Q, demand)
31         leftover = np.maximum(Q - demand, 0)
32         p = price * sales + salvage * leftover - cost * Q
33         return p.mean(), np.percentile(p, 5)
34     qs = np.arange(50, 151)
35     records = [(Q, *profit(Q)) for Q in qs]
36     return max(records, key=lambda r: r[1])
37
38 # ----- Global Optimization -----
39 def global_optimize():
40     def rastrigin(x):
41         x = np.asarray(x)
42         return 20 + np.sum(x**2 - 10 * np.cos(2 * np.pi * x))
43     bounds = [(-5.12, 5.12), (-5.12, 5.12)]
44     return differential_evolution(rastrigin, bounds, seed=0)
45
46 if __name__ == "__main__":
47     print("LP:", solve_lp().fun, solve_lp().x)
48     print("NLP:", solve_nlp().fun, solve_nlp().x)
49     print("Monte Carlo:", monte_carlo_inventory())
50     print("Global:", global_optimize().fun, global_optimize().x)

```

12 OR-Tools 0-1 规划代码

```

1 from ortools.sat.python import cp_model
2
3 def solve_binary_knapsack(values, weights, capacity):

```

```

4     model = cp_model.CpModel()
5     n = len(values)
6     x = [model.NewBoolVar(f"x{j}") for j in range(n)]
7     model.Add(sum(weights[j] * x[j] for j in range(n)) <= capacity)
8     model.Maximize(sum(values[j] * x[j] for j in range(n)))
9     solver = cp_model.CpSolver()
10    status = solver.Solve(model)
11    if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
12        return None
13    return {
14        "objective": solver.ObjectiveValue(),
15        "x": [solver.Value(var) for var in x],
16        "status": solver.StatusName(status),
17    }

```

13 CVXPY 凸优化代码

```

1  import cvxpy as cp
2  import numpy as np
3
4  # Portfolio-like convex quadratic programming
5  mu = np.array([0.08, 0.12, 0.10])
6  Sigma = np.array([
7      [0.10, 0.02, 0.01],
8      [0.02, 0.15, 0.03],
9      [0.01, 0.03, 0.12],
10 ])
11
12 x = cp.Variable(3, nonneg=True)
13 risk_aversion = 2.0
14 objective = cp.Maximize(mu @ x - risk_aversion * cp.quad_form(x, Sigma))
15 constraints = [cp.sum(x) == 1]
16 prob = cp.Problem(objective, constraints)
17 prob.solve()
18 print(prob.value, x.value)

```

14 参考资料与网页实例

1. Boyd, S. and Vandenberghe, L. *Convex Optimization*. Stanford University. 该书系统介绍凸优化，并说明线性规划和最小二乘是凸优化中的重要特例。
2. SciPy Documentation: `scipy.optimize`. 说明 SciPy 提供线性规划、非线性优化、全局优化、最小二乘等函数。
3. Google OR-Tools Documentation: Linear Programming and CP-SAT Solver. 官方文档给出 LP 求解基本步骤，并说明 CP-SAT 适合整数规划和约束规划。

4. NEOS Guide: Integer Programming. 给出整数规划标准形式与基本概念。
5. Pyomo Documentation. Pyomo 是 Python 优化建模语言，可表达线性、整数、非线性、混合整数非线性等问题。
6. CVXPY Documentation and Diamond & Boyd paper. CVXPY 是嵌入 Python 的凸优化建模语言，能按数学形式表达问题并调用求解器。
7. SciPy Documentation: `differential_evolution` and `dual_annealing`. 用于全局优化和随机启发式求解。
8. Goal Programming resources. 目标规划是多目标优化分支，通过偏差变量、加权偏差或优先等级处理多目标冲突。
9. Optuna Documentation. Optuna 是机器学习中常用的超参数优化框架，体现了现代黑箱优化和贝叶斯优化思想。